Shadow Protocol: Live Demonstration Script (7 minutes)

Before the session (10 minutes earlier)

☐ Install `release/Shadow-Protocol-1.0.0/Shadow Protocol_1.0.0_x64-setup.exe` on the demonstration laptop (per-user, no administrator rights needed) and launch it once from the Start menu.

☐ Set the screen resolution to 1920 × 1080 or 1366 × 768 and maximise the window. In **Settings**, turn the volume to a level suitable for the room (sound effects are generated by the app).

☐ Prepare a JSON file for step 7. In the Lab, load the preset **"Ends with 01 (NFA)"**, open **Save / Load → Save as .json file**, and keep the file `ends-with-01.json` in Downloads.

☐ Return to the main menu. Close other applications and notifications.

☐ Open `screenshots/` in a file viewer, minimised, in case the fallback plan is needed.

Timeline

Time	Step	What to do	What to say
0:00–0:30	**1. Introduce**	Show the main menu.	"Shadow Protocol is a stealth game whose guards are finite automata, plus tools to learn DFAs and NFAs. Everything you'll see is computed by one automata engine."
0:30–0:50	**2. Launch**	(Already open from the Start menu.) Point at the menu items.	"This is the installed Windows application. It runs offline: no server, no internet."
0:50–2:00	**3. Movement and detection**	Click ▶ **Play**. Move with WASD past Guard 1's yellow vision cone, then step into it. Press **Space** near a guard to make a noise.	"The yellow cone is the guard's vision: 90 degrees, blocked by walls. When I step in, the guard emits PLAYER_SPOTTED: once, not every frame. A noise produces SOUND_HEARD for guards within hearing range, even through walls."
2:00–3:00	**4. Live DFA transitions**	Watch the right-hand panel: the current state lights up and the last edge is highlighted. Click ⏸ **Pause**, then **Step** a few times; open **Transition history** and **Formal definition**. Click another guard to switch.	"This is Guard 1's DFA, live. Spotted: PATROL to ALERT. After the 700 ms reaction timer: ALERT to CHASE. Paused, Step advances exactly one 1/60-second tick. The formal definition shows the 5-tuple and highlights the δ cell just used. Each guard runs its own copy of the same DFA."
3:00–4:00	**5. NFA active sets**	Click the **NFA ⇄ DFA** tab. Choose the example **sound · caught**. Press **Step** twice. Point at the violet states and the trace table.	"This NFA has two transitions on 'sound' and two ε-edges. After 'sound' it is in four states at once: the ε-closure adds CHASE and PATROL. That's not randomness; every path is followed. After 'caught', one path reaches CAPTURE, so it's accepted, and the subset DFA on the right is in exactly that set."
4:00–5:00	**6. Convert NFA → DFA**	Click **Automata Lab → Load preset… → Suspicious guard NFA**. Open the **NFA → DFA** tab and click **Convert to DFA (subset construction)**.	"The subset construction builds only the reachable subsets: here 7 of a possible 32. Each DFA state is named by the set of NFA states it stands for, and the two machines agree on all 3,906 strings up to length 5."
5:00–5:40	**7. Load an automaton**	**Save / Load → Load .json file…** and choose `ends-with-01.json` (or load the preset "Ends with 01 (NFA)").	"Automata can be saved and loaded as JSON files. The file is checked when loaded, and any problems are shown in the Problems tab."
5:40–6:30	**8. Accept / reject**	**Simulate** tab: type `1001`, press **set input**, then **Run** ▸▸ → ACCEPTED. Then `0110` → REJECTED. Optionally press **Step** ▸ to show the active set grow.	"1001 ends in 01: the final active set {q0, q2} contains the accepting state, so it is accepted. 0110 ends in {q0, q1}, with no accepting state, so it is rejected."
6:30–7:00	**9. Summarise**	Click **Tutorial** and show the contents list.	"The tutorial teaches all of this in seven modules, with four challenges solved in the Lab. Every example is checked against the engine by our 527 automated tests. The automaton isn't a picture of the game: it is the game's logic."

Backup facts (if asked during the demo)

- Guard DFA: 5 states, 5 symbols, 25 transitions; q_0 = PATROL; F = {CAPTURE}.
- Vision 96 px with a 90° cone; hearing 128 px; PLAYER_LOST after 1.5 s out of sight; ALERT 0.7 s; SEARCH look-around 4 s.

- Guard NFA → DFA: 7 states. "Ends in 01": 3 states. "Third from end is 1": 8 states (proven minimal).

Fallback plan

Problem	Fallback
The application does not start	Reinstall from the installer (under a minute). If WebView2 is missing (rare on Windows 10/11), present from the screenshots instead.
Game input not responding	Click inside the game area once (the window may have lost keyboard focus), or use the header tab **Game · guard DFAs**. Press **R** to restart the level.
A guard does not react in time	Use **Pause** and **Next event**, which runs the simulation until the next DFA event; or show `screenshots/02-gameplay-guard-dfa.png`.
Lab file dialog problems	Use **Load preset… → Ends with 01 (NFA)** instead of the JSON file.
Laptop or projector failure	Present from `screenshots/` in this order: 01 (menu) → 02 and 03 (live DFA and δ table) → 04 and 05 (NFA active sets) → 06 and 07 (conversion) → 08 (accepted string) → 10 (tutorial subset construction) → 11 (challenge passed). Use `diagrams/rendered/03-guard-dfa.png` and `05c-subset-dfa-guard-nfa.png` for the theory.
Questions about testing	If a terminal is available: `npm test` in the project folder runs the 527 unit tests (a few seconds).